Tutorial: Probabilistic Inference of Roles in Memo

Welcome!

In this tutorial we will be using memo to model how agents infer their role within an institution which determines how they then act.

To achieve this, we will be modelling a situation where each agent starts off unsure which role it is supposed to play, but where all players gradually converge to a shared role assignment over the course of multiple steps by inferring the role assignment that best explains the actions they've taken so far.

Outline:

Literature search

Bayesian Delegation: (Wu et al. 2021) [1] too establish the foundation for our approach through their dynamic belief updating mechanism P(ta|H). We extend their Bayesian inference framework from task allocation priors P(ta) to institutional structure priors P(S), maintaining their interactive alignment process but applying it to role inference rather than task assignment.

Institutional Stance: (Jara-Ettinger and Dunham 2024) [2] provide theoretical grounding for why agents would prioritize structural over mental state inference. Their emphasis on role-based normative expectations directly motivates our focus on inferring institutional arrangements that constrain and enable action.

1. Basic Game

1.1 Installing and Importing Dependencies

We start first by installing memo-lang, then importing the relevant packages

```
# !pip install memo-lang
```

```
from memo import memo
import jax
import jax.numpy as np
from enum import IntEnum
```

1.2 Game Setup

We start with a simplified game which just has 3 roles and 3 actions.

Roles

Each player can adopt one of three roles:

- **Fighter**: Specializes in attacking
- **Tank**: Specializes in defense
- **Healer**: Specializes in support

Actions

Each turn, players choose one action:

- **Attack**: Deal damage to enemy
- **Defend**: Protect team from enemy attacks
- **Heal**: Restore team health

```
class Role(IntEnum): Fighter = 0; Tank = 1; Healer = 2
class Action(IntEnum): Attack = 0; Defend = 1; Heal = 2

ROLE_ACTION_PROBS = np.array([
  [0.70, 0.25, 0.05],  # fighter mostly attacks (70% attack, 25% defend, 5% heal)
  [0.25, 0.70, 0.05],  # tank mostly defends (25% attack, 70% defend, 5% heal)
  [0.25, 0.05, 0.70],  # healer mostly heals (25% attack, 5% defend, 70% heal)
])

@jax.jit
def role_policy(role, action):  # role-conditioned action likelihood P(action | role)
    return ROLE_ACTION_PROBS[role, action]
```

```
@memo
def role_inference[r0 : Role, r1 : Role, r2 : Role](role_prior:... , obs_a0, obs_a1, obs_a2):
    observer: knows(r0, r1, r2) # Push array axis variables into observer's frame
    observer: thinks[
        team: assigned(r0 in Role, r1 in Role, r2 in Role, wpp=get_element(role_prior, r0, r1, r2)), # Assign roles to ea
        team: chooses(a0 in Action, wpp=role_policy(r0, a0)), # Choose player 0's action (a0) according to their role (r0
        team: chooses(a1 in Action, wpp=role_policy(r1, a1)), # Choose player 1's action (a1) according to their role (r1
        team: chooses(a2 in Action, wpp=role_policy(r2, a2))  # Choose player 2's action (a2) according to their role (r2
    ]
    observer: observes_that [team.a0 == obs_a0] # Observe player 0's action
    observer: observes_that [team.a1 == obs_a1] # Observe player 1's action
    observer: observes_that [team.a2 == obs_a2] # Observe player 2's action
    return observer[Pr[r0 == team.r0 and r1 == team.r1 and r2 == team.r2]]

# Due to some of memo's limitations, we also need to introduce a helper function that extracts an element from a (3D) arr
@jax.jit
def get_element(array, i0, i1, i2):
    return array[i0, i1, i2]
```

```
role_prior = np.ones((3, 3, 3))
actions = [Action.Attack, Action.Defend, Action.Heal]
role_probs = role_inference(role_prior, *actions, print_table=True)

def marginalize(probs, keepaxis):
    return np.sum(probs, axis=tuple(i for i in range(probs.ndim) if i != keepaxis))

# Print the distribution over roles for each player
print(f"Distribution over roles for player 0: {marginalize(role_probs, 0)}")
print(f"Distribution over roles for player 1: {marginalize(role_probs, 1)}")
print(f"Distribution over roles for player 2: {marginalize(role_probs, 2)}")
```

```
# Create a constrained prior that ensures role uniqueness
i, j, k = np.indices((3, 3, 3))
mask = (i != j) & (i != k) & (j != k)
constrained_prior = mask.astype(np.float32)
# Run role inference and print results
role_probs = role_inference(constrained_prior, *actions, print_table=True)
```

Player 1 Action

Attack

Player 2 Action

Attack

Player 3 Action

Heal

Is Role Unique?

☐ Yes

```
import random

def simulate_role_convergence(n_steps, role_prior):
  role_probs = role_prior
  # Preallocate arrays for histories
  role_history = np.zeros((n_steps, 3), dtype=np.int32)
  action_history = np.zeros((n_steps, 3), dtype=np.int32)
  role_prob_history = np.zeros((n_steps, 3, 3, 3), dtype=np.float32)
  for t in range(n_steps):
    # Sample a role for each agent according to the marginal distribution over their role
    roles = [random.choices([0, 1, 2], weights=marginalize(role_probs, i))[0] for i in range(3)]
    role_history = role_history.at[t].set(np.array(roles)) # Store roles

    # Sample an action for each agent according to the role policy
    actions = [random.choices([0, 1, 2], weights=ROLE_ACTION_PROBS[r, :])[0] for r in roles]
    action_history = action_history.at[t].set(np.array(actions)) # Store actions

    # Update (common) beliefs about role assignments via role inference
    role_probs = role_inference(role_probs, *actions)
    role_prob_history = role_prob_history.at[t].set(role_probs) # Store updated probabilities

  return role_history, action_history, role_prob_history
```

Random Seed

5 / 100

No. of Time Steps

5 / 100

```
random.seed(random_seed)
n_steps = time_steps
role_prior = np.ones((3, 3, 3))
role_history, action_history, role_prob_history = simulate_role_convergence(n_steps, role_prior)
```

```
import matplotlib.pyplot as plt

# Create a figure with 3 subplots, one for each player
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for i in range(3): # Iterate over players
    # Calculate marginal probabilities for the current player at each timestep
    player_role_probs = [marginalize(role_prob_history[t], i) for t in range(n_steps)]
    # Add the initial timestep's probabilities
    player_role_probs.insert(0, marginalize(role_prior / role_prior.sum(), i))
    player_role_probs = np.array(player_role_probs)

    # Plot the evolution of role probabilities for the current player
    for role in Role:
        axes[i].plot(range(n_steps + 1), player_role_probs[:, role], label=role.name)

    axes[i].set_title(f'P(r{i} | actions)')
    axes[i].set_xlabel('Timestep')
    axes[i].set_ylabel('Probability')
    axes[i].legend(frameon=False, loc='upper left')
    axes[i].grid(True)

plt.tight_layout()
plt.show()
```

```
class Role(IntEnum): Fighter = 0; Tank = 1; Healer = 2;
class Action(IntEnum): Attack = 0; Defend = 1; Heal = 2;
class EnemyHealth(IntEnum): DEAD = 0;LOW = 1; MEDIUM = 2;HIGH = 3; FULL = 4;
class TeamHealth(IntEnum): DEAD = 0;LOW = 1; MEDIUM = 2;HIGH = 3; FULL = 4;
```

```
ROLE_ACTION_STATE_PROBS_V1 = np.array([
    # Fighter probabilities [enemy_health, team_health, action]
    # Base: Attack 0.90, adapts based on game state
    # - Higher attack when enemy is LOW (easier target) or team is HIGH (confident)
    # - Lower attack when enemy is HIGH (harder target) or team is LOW (more cautious)
    [
        # enemy DEAD - random regardless of team health
        [[0.33, 0.33, 0.34], [0.33, 0.33, 0.34], [0.33, 0.33, 0.34], [0.33, 0.33, 0.34], [0.33, 0.33, 0.34]],
        # enemy LOW, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.90, 0.06, 0.04], [0.90, 0.06, 0.04], [0.90, 0.06, 0.04], [0.90, 0.06, 0.04]],
        # enemy MEDIUM, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.60, 0.25, 0.15], [0.90, 0.06, 0.04], [0.90, 0.06, 0.04], [0.90, 0.06, 0.04]],
        # enemy HIGH, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.50, 0.30, 0.20], [0.70, 0.18, 0.12], [0.90, 0.06, 0.04], [0.90, 0.06, 0.04]],
        # enemy FULL, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.40, 0.35, 0.25], [0.60, 0.25, 0.15], [0.80, 0.12, 0.08], [0.90, 0.06, 0.04]],
    ],
    # Tank probabilities [enemy_health, team_health, action]
    # Base: Defend 0.90, adapts based on game state
    # - Higher defend when team is LOW (protect team) or enemy is HIGH (more threatening)
    # - Lower defend when team is HIGH (less protection needed) or enemy is LOW (less threatening)
    [
        # enemy DEAD - random regardless of team health
        [[0.33, 0.33, 0.34], [0.33, 0.33, 0.34], [0.33, 0.33, 0.34], [0.33, 0.33, 0.34], [0.33, 0.33, 0.34]],
        # enemy LOW, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.10, 0.75, 0.15], [0.15, 0.70, 0.15], [0.20, 0.65, 0.15], [0.25, 0.60, 0.15]],
        # enemy MEDIUM, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.08, 0.82, 0.10], [0.10, 0.80, 0.10], [0.15, 0.75, 0.10], [0.20, 0.70, 0.10]],
        # enemy HIGH, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.05, 0.88, 0.07], [0.06, 0.86, 0.08], [0.08, 0.84, 0.08], [0.10, 0.82, 0.08]],
        # enemy FULL, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.04, 0.90, 0.06], [0.04, 0.88, 0.08], [0.06, 0.86, 0.08], [0.08, 0.84, 0.08]],
    ],
    # Healer probabilities [enemy_health, team_health, action]
    # Base: Heal 0.90, adapts based on game state
    # - Higher heal when team is LOW (team needs healing)
    # - Lower heal when team is HIGH (team doesn't need healing)
    # - Consistent across enemy health (always focused on healing regardless of enemy state)
    [
        # enemy DEAD - random regardless of team health
        [[0.33, 0.33, 0.34], [0.33, 0.33, 0.34], [0.33, 0.33, 0.34], [0.33, 0.33, 0.34], [0.33, 0.33, 0.34]],
        # enemy LOW, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.05, 0.05, 0.90], [0.08, 0.07, 0.85], [0.10, 0.10, 0.80], [0.15, 0.15, 0.70]],
        # enemy MEDIUM, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.05, 0.05, 0.90], [0.08, 0.07, 0.85], [0.10, 0.10, 0.80], [0.15, 0.15, 0.70]],
        # enemy HIGH, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.05, 0.05, 0.90], [0.08, 0.07, 0.85], [0.10, 0.10, 0.80], [0.15, 0.15, 0.70]],
        # enemy FULL, team: [DEAD, LOW, MEDIUM, HIGH, FULL]
        [[0.33, 0.33, 0.34], [0.05, 0.05, 0.90], [0.08, 0.07, 0.85], [0.10, 0.10, 0.80], [0.15, 0.15, 0.70]],
    ],
])

@jax.jit
def role_policy_v1(role, action, enemy_health, team_health):
    """Get probability of action given role and game state."""
    return ROLE_ACTION_STATE_PROBS_V1[role, enemy_health, team_health, action]
```

```
@memo
def role_inference_v1[r0: Role, r1: Role, r2: Role](
    role_prior: ..., obs_a0, obs_a1, obs_a2, enemy_health, team_health
):
    """
    Infer role probabilities given observed actions and game state.

    Args:
        role_prior: 3x3x3 array of prior probabilities P(r0, r1, r2)
        obs_a0, obs_a1, obs_a2: Observed actions for players 0, 1, 2
        enemy_health: Current enemy health state
        team_health: Current team health state

    Returns:
        3x3x3 array of posterior probabilities P(r0, r1, r2 | actions, state)
    """
    observer: knows(r0, r1, r2)  # Push array indices into observer's frame

    # Observer's generative model
    observer: thinks[
        # Sample role assignment according to prior
        team: assigned(
            r0 in Role, r1 in Role, r2 in Role,
            wpp=get_element(role_prior, r0, r1, r2)
        ),
        # Each player chooses action according to their role policy
        team: chooses(a0 in Action, wpp=role_policy_v1(r0, a0, enemy_health, team_health)),
        team: chooses(a1 in Action, wpp=role_policy_v1(r1, a1, enemy_health, team_health)),
        team: chooses(a2 in Action, wpp=role_policy_v1(r2, a2, enemy_health, team_health))
    ]

    # Condition on observed actions
    observer: observes_that [team.a0 == obs_a0]
    observer: observes_that [team.a1 == obs_a1]
    observer: observes_that [team.a2 == obs_a2]

    # Return posterior probability of this role assignment
    return observer[Pr[r0 == team.r0 and r1 == team.r1 and r2 == team.r2]]

# Due to some of memo's limitations, we also need to introduce a helper function that extracts an element from a (3D) arr
@jax.jit
def get_element(array, i0, i1, i2):
    return array[i0, i1, i2]
```

```
def generate_player_stats(player_id, seed=42, mode='random'):
    """
    Generate player stats [STR, DEF, SUP] that sum to 1.0.

    Args:
        player_id: Unique identifier for reproducibility
        seed: Random seed
        mode: 'random' (Dirichlet), 'balanced' (0.33 each), 'specialist' (0.8+ in one stat)
    """
    random.seed(seed + player_id)

    if mode == 'balanced':
        return np.array([1/3, 1/3, 1/3])
    elif mode == 'specialist':
        # Put 0.8-0.9 in one stat, split remainder
        dominant_idx = player_id % 3
        dominant_val = 0.8 + random.random() * 0.15  # 0.8 to 0.95
        remainder = 1.0 - dominant_val
        other_val = remainder / 2
        stats = [other_val, other_val, other_val]
        stats[dominant_idx] = dominant_val
        return np.array(stats)
    else:  # 'random' - Dirichlet distribution
        alpha = [1.0, 1.0, 1.0]
        raw_stats = [random.gammavariate(a, 1.0) for a in alpha]
        total = sum(raw_stats)
        return np.array([s / total for s in raw_stats])
```

```
import jax.random as jrandom

@jax.jit
def transition(enemy_health, team_health, actions, stats, difficulty, key):
    """
    Compute next health states based on actions and stats.

    Args:
        enemy_health: Current enemy health (0-4)
        team_health: Current team health (0-4)
        actions: Array of 3 actions [player0_action, player1_action, player2_action]
        stats: Array of shape (3, 3) - each row is [STR, DEF, SUP] for a player
        difficulty: Enemy difficulty multiplier (0.5=easy, 1.0=normal, 1.5=hard)
        key: JAX random key

    Returns:
        next_enemy_health, next_team_health
    """
    # Split random key for enemy and team transitions
    key_enemy, key_team = jrandom.split(key)

    # Count actions and compute weighted sums
    attack_mask = (actions == Action.Attack).astype(np.float32)
    defend_mask = (actions == Action.Defend).astype(np.float32)
    heal_mask = (actions == Action.Heal).astype(np.float32)

    # Sum stats for each action type
    total_attack_str = np.sum(attack_mask * stats[:, 0])  # STR of attackers
    total_defend_def = np.sum(defend_mask * stats[:, 1])  # DEF of defenders
    total_heal_sup = np.sum(heal_mask * stats[:, 2])     # SUP of healers

    # ========== ENEMY HEALTH TRANSITION ==========
    # Base enemy damage taken depends on number of attacks and total STR
    # More attacks + higher STR = more likely to decrease enemy health

    # Probability enemy health decreases (gets damaged)
    base_enemy_damage_prob = 0.3 + (total_attack_str * 0.4)  # 0.3-0.7 range
    enemy_damage_prob = np.clip(base_enemy_damage_prob, 0.1, 0.9)

    # Enemy at lower health is easier to damage
    state_multiplier = 1.0 + 0.1 * (4 - enemy_health)  # 1.0-1.4x
    enemy_damage_prob = np.clip(enemy_damage_prob * state_multiplier, 0.0, 1.0)

    # Compute transition probabilities: [decrease, stay, increase]
    # Enemy rarely heals unless at DEAD (use JAX where for JIT compatibility)
    enemy_heal_prob = np.where(enemy_health > 0, 0.05, 0.0)
    enemy_stay_prob = 1.0 - enemy_damage_prob - enemy_heal_prob

    enemy_trans_probs = np.array([enemy_damage_prob, enemy_stay_prob, enemy_heal_prob])
    enemy_trans_probs = enemy_trans_probs / enemy_trans_probs.sum()  # Normalize

    # Sample transition
    enemy_delta = jrandom.choice(key_enemy, np.array([-1, 0, 1]), p=enemy_trans_probs)
    next_enemy_health = np.clip(enemy_health + enemy_delta, 0, 4)

    # ========== TEAM HEALTH TRANSITION ==========
    # Team health affected by:
    # 1. Enemy attacks (damage) - based on enemy health and difficulty
    # 2. Defend actions (mitigation) - based on total DEF
    # 3. Heal actions (recovery) - based on total SUP

    # Enemy attack strength (higher enemy health = stronger attacks)
    enemy_attack_base = np.where(enemy_health > 0, 0.4, 0.0)
    enemy_attack_str = enemy_attack_base * (1.0 + 0.15 * enemy_health) * difficulty

    # Defense mitigation (defenders reduce damage)
    defense_mitigation = total_defend_def * 0.5  # DEF reduces damage
    net_damage_prob = np.clip(enemy_attack_str - defense_mitigation, 0.0, 1.0)

    # Healing effectiveness
    heal_prob = total_heal_sup * 0.6  # SUP increases healing
    heal_prob = np.clip(heal_prob, 0.0, 0.9)

    # When team is at low health, damage is more critical (JAX-compatible conditional)
    damage_multiplier = np.where(team_health == 1, 1.2, 1.0)
    net_damage_prob = net_damage_prob * damage_multiplier

    # Compute team transition probabilities
    # Healing and damage compete
    stay_prob = 1.0 - net_damage_prob - heal_prob
    stay_prob = np.maximum(stay_prob, 0.0)

    # Normalize
    total_prob = net_damage_prob + stay_prob + heal_prob
    team_trans_probs = np.array([net_damage_prob, stay_prob, heal_prob]) / total_prob
```

```
    # Sample transition
    team_delta = jrandom.choice(key_team, np.array([-1, 0, 1]), p=team_trans_probs)
    next_team_health = np.clip(team_health + team_delta, 0, 4)

    return next_enemy_health, next_team_health
```

```
import numpy as onp

def uniform_prior_v1():
    """All role assignments equally likely."""
    prior = np.ones((3, 3, 3))
    return prior / np.sum(prior)

def utility_based_prior_v1(player_stats_list, temperature=0.5, constrained=False):
    """
    Create role prior based on player stats.

    Args:
        player_stats_list: List of 3 arrays, each [STR, DEF, SUP]
        temperature: Higher = flatter distribution (more uncertainty)
        constrained: If True, enforce unique roles (only 6 valid assignments)

    Returns:
        prior: 3x3x3 array where prior[r0, r1, r2] = P(player0=r0, player1=r1, player2=r2)
    """
    prior = np.zeros((3, 3, 3))

    for r0 in range(3):
        for r1 in range(3):
            for r2 in range(3):
                # Check uniqueness constraint if needed
                if constrained and len({r0, r1, r2}) != 3:
                    continue  # Skip duplicate role assignments

                # Utility: how well each player fits their assigned role
                u0 = player_stats_list[0][r0]  # Player 0's stat for role r0
                u1 = player_stats_list[1][r1]  # Player 1's stat for role r1
                u2 = player_stats_list[2][r2]  # Player 2's stat for role r2

                total_utility = u0 + u1 + u2
                utility = np.exp(total_utility / temperature)
                prior = prior.at[r0, r1, r2].set(utility)

    # Normalize
    return prior / np.sum(prior)

# Example: Compare priors
example_stats = [
    np.array([0.7, 0.2, 0.1]),  # Player 0: Strong fighter
    np.array([0.2, 0.7, 0.1]),  # Player 1: Strong tank
    np.array([0.1, 0.2, 0.7]),  # Player 2: Strong healer
]

uniform = uniform_prior_v1()
utility_uncon = utility_based_prior_v1(example_stats, temperature=0.5, constrained=False)
utility_con = utility_based_prior_v1(example_stats, temperature=0.5, constrained=True)

print("\nPrior comparison for specialist team:")
print(f"Uniform prior: All 27 assignments have prob {float(uniform[0,0,0]):.4f}")
print(f"\nUnconstrained utility prior (top 3):")
flat = [(r0,r1,r2, float(utility_uncon[r0,r1,r2])) for r0 in range(3) for r1 in range(3) for r2 in range(3)]
flat.sort(key=lambda x: x[3], reverse=True)
for r0,r1,r2,p in flat[:3]:
    print(f"  ({Role(r0).name}, {Role(r1).name}, {Role(r2).name}): {p:.4f}")

print(f"\nConstrained utility prior (all 6):")
flat_con = [(r0,r1,r2, float(utility_con[r0,r1,r2])) for r0 in range(3) for r1 in range(3) for r2 in range(3) if utility_
flat_con.sort(key=lambda x: x[3], reverse=True)
for r0,r1,r2,p in flat_con:
    print(f"  ({Role(r0).name}, {Role(r1).name}, {Role(r2).name}): {p:.4f}")

# Visualize the three different priors
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

priors = [
    (uniform, 'Uniform Prior'),
    (utility_uncon, 'Unconstrained Utility Prior'),
    (utility_con, 'Constrained Utility Prior')
]

for ax_idx, (prior, title) in enumerate(priors):
    ax = axes[ax_idx]

    # Extract marginal probabilities for each player
    marginals = []
    for player in range(3):
        marginal = marginalize(prior, player)
        marginals.append(marginal)

    # Plot as grouped bar chart
    x = onp.arange(3)  # Players 0, 1, 2
```

```
    width = 0.25

    for role_idx, role in enumerate(Role):
        role_probs = [marginals[player][role_idx] for player in range(3)]
        ax.bar(x + (role_idx - 1) * width, role_probs, width,
               label=role.name, alpha=0.8)

    ax.set_xlabel('Player', fontsize=11)
    ax.set_ylabel('Probability', fontsize=11)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xticks(x)
    ax.set_xticklabels([f'P{i}' for i in range(3)])
    ax.legend(title='Role', frameon=False)
    ax.set_ylim([0, 1.0])
    ax.grid(True, alpha=0.3, axis='y')

    # Add player stats as text
    stats_text = '\n'.join([f'P{i}: STR={example_stats[i][0]:.1f}, DEF={example_stats[i][1]:.1f}, SUP={example_stats[i][2
                            for i in range(3)])
    ax.text(0.02, 0.98, stats_text, transform=ax.transAxes,
            fontsize=8, verticalalignment='top', family='monospace',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.3))

plt.suptitle('Comparison of Three Prior Types: Marginal Role Probabilities by Player',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

```python
def marginalize(probs, keepaxis):
    """Extract marginal probability for a single player."""
    return np.sum(probs, axis=tuple(i for i in range(probs.ndim) if i != keepaxis))

def simulate_game_v1(
    n_steps,
    player_stats,
    role_prior,
    initial_enemy_health=EnemyHealth.FULL,
    initial_team_health=TeamHealth.FULL,
    difficulty=1.0,
    seed=42
):
    """
    Simulate the role coordination game.

    Args:
        n_steps: Maximum number of timesteps
        player_stats: List of 3 stat arrays
        role_prior: 3x3x3 prior distribution
        initial_enemy_health: Starting enemy health
        initial_team_health: Starting team health
        difficulty: Boss difficulty (0.7=easy, 1.0=normal, 1.3=hard)
        seed: Random seed

    Returns:
        Dictionary with:
        - role_history: [T x 3] sampled roles
        - action_history: [T x 3] sampled actions
        - role_prob_history: [T x 3 x 3 x 3] role beliefs
        - enemy_health_history: [T+1] enemy health states
        - team_health_history: [T+1] team health states
        - n_steps: Actual steps taken
        - outcome: 'WIN', 'LOSE', or 'DRAW'
    """
    random.seed(seed)
    key = jax.random.PRNGKey(seed)

    # Initialize
    role_probs = role_prior
    enemy_health = initial_enemy_health
    team_health = initial_team_health

    # History arrays
    role_history = []
    action_history = []
    role_prob_history = []
    enemy_health_history = [enemy_health]
    team_health_history = [team_health]

    for t in range(n_steps):
        # Sample roles from marginal beliefs
        roles = [random.choices([0,1,2], weights=marginalize(role_probs, i))[0] for i in range(3)]
        role_history.append(roles)

        # Sample actions from role policies
        actions = [
            random.choices([0,1,2], weights=ROLE_ACTION_STATE_PROBS_V1[r, enemy_health, team_health, :])[0]
            for r in roles
        ]
        action_history.append(actions)

        # Update beliefs via Bayesian inference
        role_probs = role_inference_v1(
            role_probs, *actions,
            enemy_health=enemy_health,
            team_health=team_health
        )
        role_prob_history.append(role_probs)

        # Update game state with difficulty
        key, subkey = jax.random.split(key)
        stats_array = np.array(player_stats)
        enemy_health, team_health = transition(
            enemy_health, team_health, np.array(actions), stats_array, difficulty=difficulty, key=subkey
        )

        enemy_health_history.append(enemy_health)
        team_health_history.append(team_health)

        # Check termination
        if enemy_health == EnemyHealth.DEAD:
            outcome = "WIN"
            break
        elif team_health == TeamHealth.DEAD:
```

```
            outcome = "LOSE"
            break
    else:
        outcome = "DRAW"

    return {
        'role_history': np.array(role_history),
        'action_history': np.array(action_history),
        'role_prob_history': np.array(role_prob_history),
        'enemy_health_history': np.array(enemy_health_history),
        'team_health_history': np.array(team_health_history),
        'n_steps': len(role_history),
        'outcome': outcome
    }
```

Demo Seed

33 / 100

```
# Generate player stats
DEMO_SEED = demo_seed
demo_stats = [generate_player_stats(i, seed=DEMO_SEED, mode='specialist') for i in range(3)]

print("DEMO GAME: Player Stats")
print("="*50)
for i, stats in enumerate(demo_stats):
    best_role = Role(int(np.argmax(stats))).name
    print(f"Player {i}: STR={stats[0]:.2f}, DEF={stats[1]:.2f}, SUP={stats[2]:.2f}")
    print(f"  → Best suited for: {best_role}")

# Create utility-based prior (unconstrained)
demo_prior = utility_based_prior_v1(demo_stats, temperature=0.5, constrained=False)

# Run simulation
print("\nRunning simulation...")
result = simulate_game_v1(
    n_steps=20,
    player_stats=demo_stats,
    role_prior=demo_prior,
    difficulty=1,
    seed=DEMO_SEED
)

print(f"\nResult: {result['outcome']}")
print(f"Duration: {result['n_steps']} steps")
print(f"Final Enemy Health: {EnemyHealth(int(result['enemy_health_history'][-1])).name}")
print(f"Final Team Health: {TeamHealth(int(result['team_health_history'][-1])).name}")
```

```
def compute_metrics(result, player_stats, prior):
    """
    Compute metrics for a single game result.

    Returns dictionary with:
    - outcome: 'WIN', 'LOSE', or 'DRAW'
    - duration: Number of steps
    - final_entropy: Mean entropy across players at end
    - convergence_time: First timestep where all entropies < 0.5
    - stat_alignment: % of players where final role = best stat
    """
    n_steps = result['n_steps']
    role_prob_history = result['role_prob_history']

    # Final entropy
    final_entropies = []
    for i in range(3):
        marginal = marginalize(role_prob_history[-1], i)
        final_entropies.append(entropy(onp.array(marginal)))
    final_entropy = onp.mean(final_entropies)

    # Convergence time (when all entropies < 0.1)
    convergence_time = n_steps  # Default: never converged
    for t in range(n_steps):
        entropies_t = []
        for i in range(3):
            marginal = marginalize(role_prob_history[t], i)
            entropies_t.append(entropy(onp.array(marginal)))
        if all(e < 0.1 for e in entropies_t):
            convergence_time = t
            break

    # Stat alignment
    final_roles = [onp.argmax(onp.array(marginalize(role_prob_history[-1], i))) for i in range(3)]
    best_roles = [onp.argmax(player_stats[i]) for i in range(3)]
    stat_alignment = sum(1 for i in range(3) if final_roles[i] == best_roles[i]) / 3.0

    return {
        'outcome': result['outcome'],
        'duration': n_steps,
        'final_entropy': float(final_entropy),
        'convergence_time': convergence_time,
        'stat_alignment': stat_alignment,
        'win': 1 if result['outcome'] == 'WIN' else 0,
    }
```

```
def run_multi_factor_experiment(n_games_per_condition=100, temperature=0.5, max_steps=200):
    """
    Run full multi-factor experiment: prior × stats × difficulty.

    Returns:
        DataFrame with results for each game
    """
    import pandas as pd

    prior_types = ['uniform', 'unconstrained', 'constrained']
    stat_profiles = ['random', 'balanced', 'specialist']
    difficulties = [0.5, 1.0, 1.5]  # easy, normal, hard
    difficulty_names = ['easy', 'normal', 'hard']

    results = []

    total = len(prior_types) * len(stat_profiles) * len(difficulties) * n_games_per_condition
    count = 0

    for prior_type in prior_types:
        for stat_profile in stat_profiles:
            for difficulty, diff_name in zip(difficulties, difficulty_names):
                for game_id in range(n_games_per_condition):
                    count += 1
                    if count % 20 == 0:
                        print(f"Progress: {count}/{total} games completed")

                    # Generate stats
                    seed = game_id * 1000 + hash(f"{stat_profile}_{diff_name}") % 1000
                    player_stats = [generate_player_stats(i, seed=seed, mode=stat_profile) for i in range(3)]

                    # Create prior
                    if prior_type == 'uniform':
                        prior = uniform_prior_v1()
                    elif prior_type == 'unconstrained':
                        prior = utility_based_prior_v1(player_stats, temperature, constrained=False)
                    else:  # constrained
                        prior = utility_based_prior_v1(player_stats, temperature, constrained=True)

                    # Run game with difficulty
                    result = simulate_game_v1(
                        n_steps=max_steps,
                        player_stats=player_stats,
                        role_prior=prior,
                        difficulty=difficulty,
                        seed=seed
                    )

                    # Compute metrics
                    metrics = compute_metrics(result, player_stats, prior)
                    metrics['prior_type'] = prior_type
                    metrics['stat_profile'] = stat_profile
                    metrics['difficulty'] = diff_name
                    metrics['difficulty_value'] = difficulty
                    metrics['game_id'] = game_id

                    results.append(metrics)

    print(f"\nCompleted all {total} games!")
    return pd.DataFrame(results)

# Run experiment (this takes ~3-4 minutes)
print("Starting multi-factor experiment...\n")
print("Factors: 3 priors × 3 stat profiles × 3 difficulties × 100 games = 2700 games\n")
experiment_results = run_multi_factor_experiment(n_games_per_condition=100)
print("\nExperiment complete!")
```